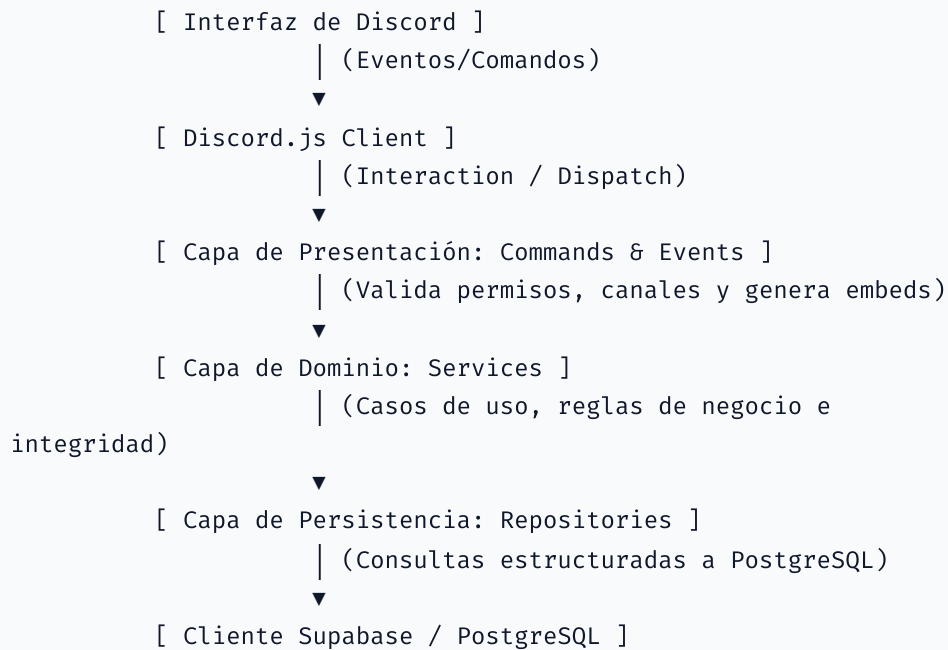


Documentación Técnica del Proyecto - La Taberna de Askhun

Esta documentación detalla la arquitectura de software, la organización de archivos, el modelo de datos físico de la base de datos (PostgreSQL/Supabase) y los flujos lógicos clave del bot de Discord **La Taberna de Askhun**.

1. Arquitectura General y Capas del Bot

El bot está diseñado siguiendo una arquitectura limpia y modular dividida en capas desacopladas, lo que permite separar la interacción externa (Discord.js) de la lógica de negocio central (Services) y de la persistencia de datos (Repositories).



1.1. Inyección de Dependencias (DI Container)

El sistema utiliza un contenedor centralizado de dependencias (`ApplicationContainer`) inicializado durante el arranque en la clase `StartupManager`. Todos los servicios, repositorios, clientes externos y configuraciones se registran aquí para evitar dependencias acopladas o variables globales.

- **Registros:** Se instancian repositorios compartiendo el mismo cliente de Supabase, y los servicios se instancian inyectando los repositorios necesarios.
- **Acceso Tipado:** El contenedor provee propiedades de lectura directa (`container.supabase`, `container.userRepository`, etc.) facilitando el autocompletado y evitando la resolución manual de tipos por string en gran parte del código.

1.2. Desacoplamiento de Lógica de Negocio

Los comandos de Discord (`src/commands/`) actúan exclusivamente como **controladores de presentación**. Su responsabilidad se limita a:

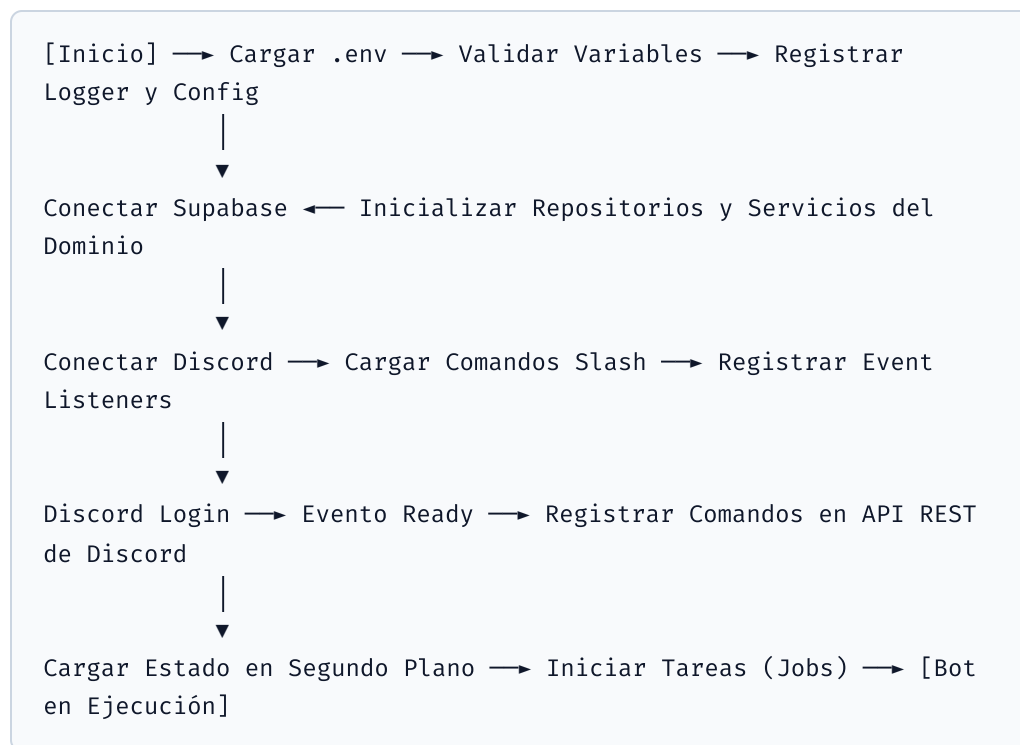
- Extraer los parámetros de la interacción.
- Llamar a los servicios del dominio pasándoles parámetros primitivos (IDs de Discord, cantidades, nombres de artículos).

- Capturar errores del dominio (ej. saldo insuficiente, requisitos no cumplidos) y formatear la respuesta visual mediante mensajes interactivos o embeds de Discord.

Los servicios (`src/services/`) procesan las transacciones y validan las reglas de negocio de manera agnóstica a Discord, lo que permitiría integrar fácilmente un panel web o una API REST en el futuro utilizando los mismos servicios y base de datos.

2. Inicialización y Ciclo de Vida del Bot (Bootstrapping)

La inicialización del bot se gestiona de forma secuencial y estructurada a través de las fases controladas por `StartupManager` :



2.1. Secuencia de Arranque Detallada

1. **Carga de Entorno (`EnvironmentLoader`)**: Lee el archivo `.env` del proyecto.
2. **Validación Estricta (`EnvironmentValidator`)**: Comprueba la presencia y formato de variables obligatorias (`DISCORD_TOKEN` , `DISCORD_CLIENT_ID` , `SUPABASE_URL` , `SUPABASE_KEY`). Lanza excepciones de parada inmediata si falta algún elemento crucial.

3. **Registro de Configuración y Logs:** Registra en el contenedor `ConfigurationService` y `Logger`.
 4. **Conexión con Supabase y Registro de Repositorios/Servicios:** Instancia el cliente de Supabase y registra todos los repositorios y servicios principales del dominio en el contenedor.
 5. **Instanciación del Cliente de Discord:** Crea la sesión del cliente de Discord aplicando los intents requeridos (como `Guilds` y `GuildMembers`).
 6. **Carga Dinámica de Comandos (`CommandLoader`):** Escanea recursivamente el directorio `src/commands/`, filtra archivos auxiliares o interfaces, e inicializa dinámicamente las clases que implementen la interfaz `Command`, registrándolas en el `CommandRegistry`.
 7. **Carga Dinámica de Eventos (`EventLoader`):** Escanea recursivamente el directorio `src/events/`, inicializa los escuchadores de eventos y los asocia al cliente de Discord (`discord.on` o `discord.once`).
 8. **Inicio de Tareas en Segundo Plano (`JobInitializer`):** Escanea y carga los archivos de la carpeta `src/jobs/` registrándolos en el `JobScheduler`.
 9. **Inicio de Sesión en Discord (Login):** Llama a `discord.login(token)`. Si el token comienza con `mock_`, el sistema simula el inicio de sesión para entornos de pruebas locales sin conexión a internet.
 10. **Despliegue de Comandos Slash (`ReadyEvent`):** Al conectarse exitosamente a Discord, el bot lee la estructura JSON de los comandos registrados en `CommandRegistry` y utiliza el cliente REST nativo de Discord para realizar una petición HTTP PUT global a `Routes.applicationCommands(clientId)`. Esto actualiza de forma automática e instantánea los comandos Slash visibles para los usuarios de Discord.
 11. **Arranque de Jobs (`JobScheduler.startAll`):** Inicia los bucles de ejecución de las tareas en segundo plano.
-

3. Sistema de Enrutamiento y Despacho de Comandos / Eventos

3.1. Enrutamiento en `interactionCreate`

El evento `interactionCreate` actúa como el despachador de entrada de comandos y clics en botones:

- **Comandos de Barra (Slash Commands):** Busca el comando por su nombre en el `CommandRegistry`. Si existe, realiza dos validaciones estrictas antes de

ejecutarlo:

1. **Permisos del Usuario:** Consulta a través de `UserRepository.getById` si el usuario de Discord cuenta con las banderas `is_admin` o `is_owner` en la base de datos si el comando es administrativo.
 2. **Aislamiento de Canales:** Los comandos administrativos solo pueden ejecutarse en el canal con nombre exacto `askhun-moderation` (se valida convirtiendo el nombre a minúsculas). Para comandos de juego (compras, ranking, oración), el bot recupera la configuración de canales del servidor (`guild_settings`) y valida que la interacción ocurra en el canal vinculado a dicha funcionalidad (o en hilos secundarios pertenecientes a dicho canal).
- **Interacciones de Botones (Button Interactions):** Las apuestas interactúan directamente con botones. El enrutador intercepta los clics en botones (que contienen metadatos en su `customId` como `vote_predictionId_optionId` o `page_predictionId_pageIndex`), realiza comprobaciones de estado de la apuesta en base de datos e invoca el método correspondiente del `PredictionService`.
-

4. Arquitectura de Tareas en Segundo Plano (Jobs)

El bot cuenta con tareas asíncronas automáticas coordinadas por `JobScheduler`:

4.1. Expiración y Cierre de Pronósticos (`PredictionExpirationJob`)

- **Ejecución:** Comprobación en bucle cada 30 segundos (`setInterval`).
- **Lógica:**
 1. Consulta a la base de datos si existe alguna apuesta activa (`status = 'open'`) cuya fecha límite de finalización (`ends_at`) sea menor al tiempo actual (`Date.now()`).
 2. Si la encuentra, actualiza atómicamente el estado en base de datos a `'closed'`.
 3. Llama a la API de Discord para editar el mensaje del canal de apuestas, reemplazando los botones de votación por texto y notificando a los usuarios del cierre de urnas.

4.2. Mantenimiento de Conexión de Base de Datos

(SupabaseKeepAliveJob)

- **Ejecución:** Se dispara cada 5 horas de forma automatizada.
- **Lógica:**
 - Realiza una consulta muy ligera a la base de datos (`SELECT id FROM profiles LIMIT 1`).
 - Su único propósito es simular actividad de lectura recurrente, evitando que Supabase catalogue el proyecto como inactivo y suspenda el contenedor de la base de datos por inactividad.

5. Estructura de la Base de Datos

La persistencia de datos reside en PostgreSQL (gestionado mediante Supabase). El modelo físico consta de las siguientes tablas, relaciones y restricciones:

5.1. Tabla: `profiles`

Almacena los perfiles de los usuarios y su estado dentro del sistema del bot.

```
CREATE TABLE profiles (  
  id VARCHAR(255) PRIMARY KEY, -- ID de usuario de Discord  
  balance INTEGER NOT NULL DEFAULT 0, -- Saldo actual (Celesios)  
  total_spent INTEGER NOT NULL DEFAULT 0, -- Gasto histórico  
  acumulado  
  welcome_received BOOLEAN NOT NULL DEFAULT FALSE, -- Control de  
  regalo de bienvenida  
  is_admin BOOLEAN NOT NULL DEFAULT FALSE, -- Rol administrativo  
  del bot  
  is_owner BOOLEAN NOT NULL DEFAULT FALSE, -- Rol de propietario  
  supremo  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

5.2. Tabla: `transactions`

Libro mayor contable para la auditoría de todos los cambios de saldo económico en la plataforma.

```

CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id VARCHAR(255) NOT NULL REFERENCES profiles(id) ON
DELETE CASCADE,
  amount INTEGER NOT NULL, -- Valor de la transacción (positivo
o negativo)
  type VARCHAR(100) NOT NULL, -- Tipos: 'welcome', 'food',
'drink', 'title', 'admin_give', 'admin_remove',
'prediction_reward'
  reason VARCHAR(255), -- Detalle aclaratorio
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_transactions_user_id ON transactions(user_id);

```

5.3. Tabla: items

Catálogo de productos de comida y bebida disponibles para la venta.

```

CREATE TABLE items (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  type VARCHAR(50) NOT NULL CHECK (type IN ('food', 'drink')),
  price INTEGER NOT NULL CHECK (price ≥ 0),
  active BOOLEAN NOT NULL DEFAULT TRUE,
  message TEXT, -- Mensaje inmersivo de entrega personalizado
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT unique_active_item_name_type UNIQUE (name, type)
);
CREATE INDEX idx_items_name_type ON items(name, type);

```

5.4. Tabla: titles

Catálogo de oraciones y títulos religiosos/nobles disponibles en el templo.

```
CREATE TABLE titles (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) UNIQUE NOT NULL,
  cost INTEGER NOT NULL CHECK (cost ≥ 0),
  required_title_id INTEGER REFERENCES titles(id) ON DELETE SET
  NULL, -- Rango requisito anterior
  role_id VARCHAR(255) NOT NULL, -- ID del rol de Discord
  asociado
  active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

5.5. Tabla: **user_titles**

Tabla de unión (relación muchos a muchos) que almacena la progresión de títulos que posee cada usuario.

```
CREATE TABLE user_titles (
  user_id VARCHAR(255) NOT NULL REFERENCES profiles(id) ON
  DELETE CASCADE,
  title_id INTEGER NOT NULL REFERENCES titles(id) ON DELETE
  CASCADE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, title_id)
);
CREATE INDEX idx_user_titles_user_id ON user_titles(user_id);
```

5.6. Tabla: **predictions**

Almacena las apuestas / pronósticos del servidor creados por la administración.


```
CREATE TABLE predictions (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  reward INTEGER NOT NULL CHECK (reward ≥ 0),
  status VARCHAR(50) NOT NULL DEFAULT 'draft' CHECK (status IN
('draft', 'open', 'closed', 'resolved', 'cancelled')),
  created_by VARCHAR(255) NOT NULL REFERENCES profiles(id),
  winner_option_id INTEGER, -- Asignado al resolverse la
predicción
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  closed_at TIMESTAMP WITH TIME ZONE,
  CONSTRAINT fk_winner_option FOREIGN KEY (winner_option_id)
REFERENCES prediction_options(id) ON DELETE SET NULL
);
```

5.7. Tabla: prediction_options

Opciones asociadas a cada pronóstico.

```
CREATE TABLE prediction_options (
  id SERIAL PRIMARY KEY,
  prediction_id INTEGER NOT NULL REFERENCES predictions(id) ON
DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_prediction_options_prediction_id ON
prediction_options(prediction_id);
```

5.8. Tabla: prediction_votes

Votos de los usuarios en los pronósticos (clave primaria compuesta para asegurar un voto único por usuario y pronóstico).

```

CREATE TABLE prediction_votes (
    prediction_id INTEGER NOT NULL REFERENCES predictions(id) ON
DELETE CASCADE,
    user_id VARCHAR(255) NOT NULL REFERENCES profiles(id) ON
DELETE CASCADE,
    option_id INTEGER NOT NULL REFERENCES prediction_options(id)
ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (prediction_id, user_id)
);
CREATE INDEX idx_prediction_votes_prediction_id ON
prediction_votes(prediction_id);
CREATE INDEX idx_prediction_votes_user_id ON
prediction_votes(user_id);

```

5.9. Tabla: `guild_settings`

Configuración particular y vinculación dinámica de canales e imágenes del bot para cada servidor.

```

CREATE TABLE guild_settings (
    guild_id VARCHAR(255) PRIMARY KEY,
    taberna_channel_id VARCHAR(255),
    templo_channel_id VARCHAR(255),
    ranking_thread_id VARCHAR(255),
    pronosticos_channel_id VARCHAR(255),
    recuerda_channel_id VARCHAR(255),
    welcome_celesios INTEGER NOT NULL DEFAULT 15,
    gram_avatar_url VARCHAR(1000),
    grum_avatar_url VARCHAR(1000),
    gram_msg_not_found VARCHAR(1000),
    gram_msg_no_money VARCHAR(1000),
    grum_msg_not_found VARCHAR(1000),
    grum_msg_no_money VARCHAR(1000),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

5.10. Tabla: `tavern_phrases`

Pool de diálogos y frases aleatorias empleadas por los personajes (Gram, Grum, Oración) durante compras y eventos.

```
CREATE TABLE tavern_phrases (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    guild_id VARCHAR(255) NOT NULL,  
    character VARCHAR(50) NOT NULL, -- 'gram', 'grum', 'oracion'  
    event VARCHAR(100) NOT NULL, -- 'no_existe', 'sin_saldo',  
    'ya_obtenido', 'progreso_invalido'  
    phrase TEXT NOT NULL,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

6. Flujos Lógicos Principales

6.1. Flujo de Bienvenida de Miembros

1. El usuario entra al servidor, disparando el evento `guildMemberAdd`.
2. El bot busca si el usuario existe en `profiles`.
3. Si no existe:
 - Crea el perfil del usuario asignándole el saldo inicial configurado (ej: 15 Celesios).
 - Registra una transacción del tipo `'welcome'`.
 - Obtiene la configuración de canales del servidor (`guild_settings`) para resolver enlaces dinámicos (como el canal de Recuerda).
 - Envía al usuario un mensaje directo (DM) con un texto inmersivo y de bienvenida.
4. Si ya existía, no realiza ninguna acción comercial, evitando el abuso por reingresos repetidos.

6.2. Compra en la Taberna (`/gram` y `/grum`)

1. El comando valida que la ejecución ocurra en el canal vinculado a `Taberna` (o en un hilo hijo del mismo).
2. Se obtiene el artículo del `ItemRepository` (buscando por nombre y tipo).
3. Se comprueba si el usuario dispone de saldo suficiente en la base de datos.
4. Si no tiene saldo o no existe el producto:
 - Se comprueba si existen frases personalizadas en `tavern_phrases` para el evento. De ser así, se selecciona una frase aleatoria de la pool.
 - En su defecto, se aplica el mensaje predeterminado.

5. Si la compra es válida:
 - Se reduce el saldo del usuario de forma atómica en `profiles`.
 - Se suma el costo al campo `total_spent` (necesario para el Ranking).
 - Se registra la transacción en el ledger (`transactions`).
 - Se responde al usuario con un embed inmersivo utilizando el mensaje del producto y el avatar configurado del personaje.

6.3. Ciclo de Vida de Pronósticos

1. **Creación:** El administrador ejecuta `/pronostico crear`. Se inserta el registro en `predictions` (estado `open`), las opciones en `prediction_options` y se genera un embed interactivo con botones en el canal de Pronósticos.
2. **Votación:** Los usuarios votan pulsando los botones del embed. Se comprueba que la predicción esté `open` y que el usuario no haya votado previamente (`prediction_votes`). El voto se inserta de forma inmutable. Se actualiza el embed mostrando el censo de participantes (paginado de 15 en 15).
3. **Cierre:** Ocurre automáticamente al expirar el temporizador (`predictionExpirationJob` comprobando intervalos cada 30 segundos) o mediante ejecución manual administrativa (`/pronostico cerrar`). El estado cambia a `closed` y se eliminan los botones de votación del embed de Discord para impedir nuevos votos.
4. **Resolución:** El administrador ejecuta `/pronostico resolver posicion:[N]`.
 - Se determina la opción ganadora de la lista.
 - Se filtran los votantes ganadores en `prediction_votes`.
 - Se otorga la recompensa en Celesios a cada ganador en `profiles` y se guarda la transacción de recompensa.
 - Se actualiza el embed indicando la resolución final.